

Solaris Health (luca-passport) — Production Readiness Audit

Auditor role: Adversarial Senior Security Engineer / Lead SRE

Mode: Analysis only. No code was modified.

Date: 2026-07-22

Scope: /home/ubuntu/luca-passport — the live “Solaris Health” app (React/Vite + Node/Express + PostgreSQL, deployed via Docker Compose behind nginx at solaris-health.abacusai.cloud). The other git repos in the home dir (bittasker-*, gaswatch , arbitrum-usdt-escrow , nostr-relay-staging , and the empty solaris-health remote-mirror repo) are separate projects and were **not** in scope; this audit covers the product currently running.

Bottom line up front: this is a **feature-complete demo running on scaffolding**, not a production system. It ships live secrets in git, has zero network-edge hardening, no graceful lifecycle, no CI, and a database with no backups on a single disposable VM. Do not put real patient data near it in its current state.

PART 1 — Adversarial Security & Vulnerability Audit

1.1 CRITICAL — Production secrets committed to the repository

.env.production is **tracked in git** (git ls-files confirms) and contains real, usable secrets:

```
# .env.production (committed)
DB_PASSWORD=e6835ac2a1f099d254f451d8bfb71920
JWT_SECRET=81484de6f178dfc2aac3aa88483d9c524d278637b4bc2c043263a1db691a4f5e
API_URL=https://solaris-health.abacusai.cloud
NODE_ENV=production
```

Anyone with repo read access owns the production JWT signing key — they can **forge a valid token for any user, including admins**, and they have the production DB password. This is the single worst finding in the codebase.

1.2 CRITICAL — Secrets also live in git history

backend/.env and root .env were committed in earlier phases (git log --all -- backend/.env shows commits ed9c664 , 15e03b5 , 0000d7a). The dev JWT secret

luca_dev_secret_2026_sovereign_health is permanently in history. Removing the files now does **not** remove them from history — history rewrite + rotation is required. The currently-untracked

backend/.env on disk also holds a live platform key: LUCA_AI_API_KEY / ABACUS_API_KEY = s2_7aa303265ba04d62af532a66384369c3 .

1.3 CRITICAL — Hardcoded JWT fallback secret

backend/src/middleware/auth.js:3 :

```
const JWT_SECRET = process.env.JWT_SECRET || 'dev-secret-key';
```

If `JWT_SECRET` is ever unset (a single misconfigured deploy), the app silently signs and verifies tokens with the world-known string `'dev-secret-key'`. Tokens are issued with a **30-day expiry** and there is no revocation list — one leak is valid for a month. The same weak default (`luca_prod_secret_2026_change_in_production`) is baked into `docker-compose.yml` as a fallback.

1.4 HIGH — Wide-open CORS

`backend/src/server.js:54` :

```
app.use(cors());
```

Default `cors()` reflects **any** origin. Combined with token-in-header auth this is less catastrophic than credentialed cookies, but it means any website on the internet can script the API on behalf of a visitor who pastes/holds a token, and there is no origin allow-list.

1.5 HIGH — No rate limiting anywhere at the edge

No `express-rate-limit`, `helmet`, or `csrf` in `backend/package.json` (deps: `archiver`, `bcryptjs`, `cors`, `dotenv`, `ethers`, `express`, `jsonwebtoken`, `pg` only). Consequences:

- `POST /api/auth/login` and `/api/auth/register` are unthrottled → credential stuffing / brute force / signup spam.
- `express.json({ limit: '15mb' })` (`server.js:55`) with no rate limit → trivial memory-pressure DoS by POSTing 15 MB bodies in a loop.
- No `helmet` → missing baseline security headers (HSTS, X-Content-Type-Options, frame options, etc.).
- The only “rate limiting” in the codebase is app-level (`provider-application.js` 1/day) and an external-API courtesy limiter in `geocoding.js` — neither protects the auth surface.

1.6 MEDIUM — Unauthenticated data-exposure endpoints

`backend/src/routes/organizations.js` mounts **no auth** on `GET /api/organizations` and `GET /api/organizations/:id`. The detail route returns steward PII pulled from `users` :

`display_name`, `full_name`, `nostr_npub`, `role`. Org list + steward real names are publicly enumerable without a token. Intentional “public map pins,” but it leaks real names and identity handles.

1.7 MEDIUM — SIWE wallet proof has no replay protection

`backend/src/routes/wallet.js` : `GET /nonce` generates `crypto.randomBytes(16)` but **never persists it**. `POST /verify-signature` accepts client-supplied `message`, `signature`, `address` and only checks `ethers.verifyMessage()` recovers the address (`lib/web3.js:255`). The nonce is decorative — it is not stored, bound to the user, or marked used, so a captured signature can be replayed and the signed message is entirely attacker-chosen. (Mitigating factor: all wallet routes require `authMiddleware` + patient role, so the attacker must already own the account.)

1.8 LOW/positive notes

- **SQL injection: not found.** Queries use parameterized `$1` placeholders throughout; the one dynamic `UPDATE ... SET ${sets}` in `provider/bookings.js:111` builds column names from a server-side whitelist, values stay parameterized. Good.
- **Role escalation at signup: closed.** `auth.js` hardcodes `role = 'patient'` and explicitly ignores `req.body.role`. Admin routes gate via `router.use(authMiddleware, adminOnly)`.
- **Password hashing:** `bcryptjs` cost 10 — acceptable. But there is **no password strength policy** (no min length/complexity check on register).
- **Client key custody:** `src/lib/encryption.js` stores the user’s private messaging key in `local-Storage` unencrypted — any XSS exfiltrates it permanently.

- **Credential hygiene:** a live GitHub OAuth token (`ghu_...`) is sitting in plaintext in `.git/config` 's remote URL on the VM.
- **Web3 gas-drain vector:** N/A — on-chain payments are simulated (`payments-sim.js` , “no real money moves”); no signing of real transactions server-side, so no gas-drain surface today.

PART 2 — Infrastructure & Hosting Audit

2.1 CRITICAL — Single ephemeral VM, no production platform

The stack runs as three `docker compose` containers (`luca-passport-frontend/backend/postgres`) on **one Abacus sandbox VM**, reverse-proxied by `nginx` (`/etc/nginx/conf.d/solaris-health.conf`). There is **no** Railway/Render/Fly/Cloudflare/managed-Kubernetes anywhere. This is a single point of failure with no redundancy, no horizontal scale, and no autohealing beyond Docker's `restart: unless-stopped` . If the VM is reclaimed, the product and its data disappear.

2.2 CRITICAL — Database is an unmanaged local container with no backups

Postgres is `postgres:15-alpine` writing to a local Docker volume `pgdata` . There is **no managed database, no automated backups, no replication, no PITR**. The DB password is hardcoded in `docker-compose.yml` (`POSTGRES_PASSWORD: luca_dev_2026`) and Postgres is exposed on the host at `0.0.0.0:5432` . A VM loss or volume corruption = total, unrecoverable data loss.

2.3 HIGH — No graceful shutdown / lifecycle handling

`backend/src/server.js` calls `app.listen()` and nothing else — there is **no `process.on('SIGTERM')` / `SIGINT` handler, no `server.close()` , no `pool.end()`** at runtime (grep confirms these only appear in one-off seed scripts). On `docker stop` /redeploy the Node process (running as PID 1 via `CMD ["node", "src/server.js"]`) is hard-killed: in-flight requests are dropped and DB connections are not drained. No `dumb-init` / `tini` either, so signal handling is fragile even if a handler existed.

2.4 HIGH — Database migrations are not applied automatically

Schema is loaded **only on a fresh volume** via `docker-entrypoint-initdb.d` (the five `schema*.sql` mounts in compose). The 13 files in `backend/migrations/` (`001...013`) have **no runner** — no `node-pg-migrate` / `knex` / `Flyway`, no `migrate` npm script. New migrations must be hand-applied with `psql` and drift silently between environments. (Prior sessions already had to run migrations manually against the live DB.)

2.5 HIGH — Connection pool will crash the process on a transient error

`backend/src/db.js` :

```
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
pool.on('error', (err) => { console.error(...); process.exit(-1); });
```

- No `max` , no `idleTimeoutMillis` , no `connectionTimeoutMillis` , no `ssl` . Default pool size is 10; any future second instance would blow the hosted-PG 25-connection ceiling.
- `process.exit(-1)` on an **idle-client** error means one transient network blip **takes the whole API down** — the opposite of resilience.

2.6 MEDIUM — No structured logging or observability

No `winston` / `pino` / `morgan` (not in deps). Logging is bare `console.log` / `console.error(err.stack)` — no levels, no request/correlation IDs, no JSON logs to ship anywhere. `/api/health` (DB check) and `/api/metrics` (Prometheus text) endpoints exist and are decent, but nothing scrapes them and `/api/metrics` is unauthenticated (leaks process memory/uptime).

2.7 MEDIUM — No CI/CD

`.github/` contains only `ISSUE_TEMPLATE/` — **no workflows/**. Nothing runs lint/tests/build on push; deploys are manual `docker compose` rebuilds on the VM (backend source is baked into the image with no volume mount, so every code change needs a manual rebuild). No image scanning, no dependency audit gate.

2.8 LOW

- Dockerfile.backend runs as **root** (no `USER node`), no `HEALTHCHECK` for the backend image (compose healthchecks only Postgres).
- `restart: unless-stopped` + enabled `docker.service` gives basic restart-on-reboot, but there is no systemd unit owning `docker compose up`.

PART 3 — QA & Testing Reality Check

3.1 Coverage is thin and concentrated

- **Backend: 6 test files for 40 route files** (`tests/auth|export|luca|timeline|trends|wallet.test.js`). That is ~15% of routes touched, and only at a happy-path integration level.
- **Frontend: 4 test files** (`api` , `WalletConnect` , `HealthTimeline` , `LucaPassport`) against a much larger component/page tree.

3.2 Critical paths with ZERO automated tests

The money, booking, and consent flows — the things that would actually hurt users if broken — are untested: `payments-sim` (**split-policy math, wallet crediting**), `bookings / provider/bookings` (**state machine**), `consent` , `journeys` , `marketplace` , `provider-application /approvals`, `admin/*` , `messages` , `health-documents` , `organizations` , `gps` , `appointments` .

3.3 Tests depend on a live shared database, so they can't gate CI

`backend/tests/setup.js` loads real `backend/.env` and points at `postgresql://...luca_prod_2026@localhost:5432/luca_passport`; `auth.test.js` uses `supertest` against the **real app + real Postgres**, inserting and cleaning up rows in the dev DB. There is no mocked/ephemeral DB, so tests only pass where a seeded Postgres happens to be running — which is exactly why there is no CI running them. A stale `backend/coverage/` (lcov/clover) exists but is not wired to any gate or threshold.

3.4 “Real device / real dependency” gaps needing human QA

- **AI (LUCA):** provider defaults to a **mock** (`mode: mock` , graceful rules-based fallback in `luca-recommendations.js`). Tests and much of the UX exercise the mock, **not** the real cloud LLM path — LLM latency, token limits, and error handling are effectively untested.
- **No passkey/WebAuthn/biometric flow exists** in the codebase (the audit prompt's “mock passkey” example does not apply here). What does exist is **browser-generated encryption keys in localStorage** (`src/lib/encryption.js`) — the real risks there (key loss on cache clear, multi-device sync, XSS theft) are entirely unverified and need human QA.

- **Wallet/SIWE, GPS geolocation, base64 document upload, payment receipt animation** are UI/UX assumptions validated (at best) by the 10-step `backend/scripts/smoke-test.js`, which is a linear happy-path script, not a test suite (register → me → skip-onboarding → recommendations → check-in → week-strip → journal → audio → export).

PART 4 — The Reality Score & Prioritized Action Plan

4.1 Honest Production Readiness Score

~30%

It builds, runs, and demos end-to-end — that is real and it's why it isn't lower. But “production” means safe with real users' health data under load and failure, and on that bar it fails at the security edge, the data-durability layer, and the release process simultaneously. This is a strong **prototype/MVP**, not a production service.

4.2 Top 3 Disqualifiers (fix these or nothing else matters)

1. **Live production secrets committed to git** (`.env.production` with real `JWT_SECRET` + `DB_PASSWORD`, plus secrets in history and a platform API key on disk). Instant account-forgery and DB compromise for anyone with repo access.
2. **Zero data durability** — unmanaged single-container Postgres on one disposable VM with no backups/replication. One VM loss = permanent, total loss of all patient data.
3. **No network-edge hardening** — wide-open CORS + no rate limiting + no helmet + a hardcoded `'dev-secret-key'` JWT fallback + 30-day non-revocable tokens. The auth surface is open to brute force, forgery, and DoS.

4.3 Prioritized Action Plan to reach 80%+

Priority 1 — Contain the secret exposure (do today)

1. Rotate **everything** now: generate a new 256-bit `JWT_SECRET`, new DB password, and revoke/rotate the `ABACUS_API_KEY` / `LUCA_AI_API_KEY` (`s2_7aa3...`) and the GitHub `ghu_...` token in `.git/config`.
2. `git rm --cached .env.production`, delete it from the working tree, and purge `.env*` from history (`git filter-repo --path .env --path backend/.env --path .env.production --invert-paths`), then force-push. Confirm `.gitignore` already lists `.env / backend/.env` (it does) and add `.env.production`.
3. Move all secrets to injected environment/secret store (VM metadata or the hosting platform's secret manager). Never commit any `.env`.

Priority 2 — Close the JWT + auth-edge holes

4. Remove the `|| 'dev-secret-key'` fallback in `auth.js`; **fail fast** (throw on boot) if `JWT_SECRET` is missing. Do the same for the compose fallbacks.
5. Reduce token lifetime (e.g. 24h access token + refresh flow) and add a revocation/ `tokenVersion` check.
6. `npm i helmet express-rate-limit` in backend: `app.use(helmet())`, a global limiter, and a stricter limiter on `/api/auth/*`. Enforce a password policy on register.

Priority 3 — Lock down CORS and exposure

7. Replace `app.use(cors())` with an explicit origin allow-list (`origin: ['https://solaris-`

`health.abacusai.cloud'] , credentials only if needed).`

8. Put `/api/metrics` behind auth or bind it to localhost for the scraper only. Decide intentionally what `GET /api/organizations` exposes and strip steward `full_name` from the public payload.

9. Persist and verify SIWE nonces server-side (store per-user, single-use, TTL) and reject client-supplied messages that don't match a server-issued nonce.

Priority 4 — Give the data a home that survives

10. Migrate Postgres to a **managed** instance (Abacus hosted PG, or Neon/RDS/Railway PG) with automated daily backups + PITR. Stop exposing `5432` publicly; require SSL in the pool.

11. Harden `db.js`: set `max` ($\leq$ the hosted 25-conn ceiling), `idleTimeoutMillis`, `connectionTimeoutMillis`, `ssl`; **remove** `process.exit(-1)` on pool error — log and let the pool recover.

Priority 5 — Make the app survive restarts and deploys

12. Add a `SIGTERM` / `SIGINT` handler that stops accepting connections (`server.close()`), drains, then `pool.end()` and exits. Add `tini` / `dumb-init` (or `node --enable-source-maps` under an `init`) as PID 1 in `Dockerfile.backend`; add a `USER node` and a container `HEALTHCHECK`.

13. Add a real migration runner (`node-pg-migrate` or `knex`) with a `migrate` script run automatically on deploy; stop relying on `docker-entrypoint-initdb.d`.

Priority 6 — Observability

14. Add structured logging (`pino` + `pino-http`) with request IDs and levels; ship logs and scrape `/api/health` + `/api/metrics` with real alerting.

Priority 7 — Deploy to a real platform with CI/CD

15. Move off the single sandbox VM to a managed host (Railway/Render/Fly/Cloudflare + the managed DB above), ideally ≥ 2 instances behind the LB.

16. Add `.github/workflows/ci.yml`: `lint` $\rightarrow$ `test` (against an ephemeral Postgres service container) $\rightarrow$ `build` $\rightarrow$ `image scan` / `npm audit`, blocking merge; then CD on main.

Priority 8 — Testing reality

17. Refactor tests to run against a disposable/ephemeral Postgres (Docker service in CI or Testcontainers) instead of the shared dev DB; add a coverage threshold gate.

18. Write tests for the untested critical paths — **payments split math, booking state machine, consent, journeys, provider approval, messages** — before adding features on top of them.

19. Add the real-LLM path (not just mock) to an integration test with timeout/error handling, and get human QA sign-off on wallet/SIWE, localStorage key custody (loss/multi-device), and document upload on real devices.

Land Priorities 1–5 and you're roughly production-viable (~70%). Add 6–8 and you're at a defensible 80%+.

All findings above were verified by reading the referenced files this session. Anything not directly observed is labeled as a mitigating factor or assumption. No source files were modified.